

This assignment is **due on May 21** and should be submitted on Gradescope. All submitted work must be *done individually* without consulting someone else's solutions in accordance with the University's "Academic Integrity" policies.

As a first step go to the last page and read the section: "Advice on how to do the assignment."

Problem 1. (50 points)

As an algorithm theorist working for database technology startup QDijkstra, you have recently been working on determining the tractability of GQL queries which search for paths optimizing over multiple metrics. One such query might ask for a route from Bathurst to Sydney with total transit time at most some value T , which is *also* highly scenic, which we model as having a combined "scenery score" at least as high as some value S .

```
MATCH (u:City WHERE u.name="Bathurst")
  -[r]->*
  (v:City WHERE v.name="Sydney")
  WHERE SUM(r.travelTime)<=T
  AND SUM(r.sceneryScore)>=S
```

You have reformulated the task of evaluating this query as an algorithm design problem, which you refer to as the SCENIC-ROUTE problem: formally, given an undirected simple connected graph $G = (V, E)$, two vertices $u, v \in V$, a *travel time* function $t : E \rightarrow \mathbb{Z}_{>0}$, a *scenery score* function $s : E \rightarrow \mathbb{Z}_{>0}$, a positive integer *target time* T , and a positive integer *target scenery score* S , determine whether there exists a vertex path $p = p_1, p_2, \dots, p_k$ through G from u to v , such that

$$\sum_{0 \leq i < k} t(p_i, p_{i+1}) \leq T$$

and

$$\sum_{0 \leq i < k} s(p_i, p_{i+1}) \geq S.$$

Note that since a GQL query returns actual *path bindings*, it also makes sense to restrict the space of paths under consideration, so that only paths consisting of a reasonable amount of vertices are under consideration. For that reason, you impose the additional constraint that $k \leq n + m$, where n is the number of vertices in G and m is the number of edges in G .

In **Assignment 2 problem 2**, it was established that the SCENIC-ROUTE problem can be solved in $O((n + m)T)$ time. Being well-versed in algorithm theory, the value T appearing in this time bound bugs you: as T can be arbitrarily large, the value T appearing in the time bound means that your algorithm has *pseudopolynomial* time complexity. A natural pessimist, you suspect that this pseudopolynomiality is likely to be inherent to the SCENIC-ROUTE problem, and thereby to the original query. To provide evidence of this claim, you set out to establish that the SCENIC-ROUTE problem is **NP**-complete.

- a) Prove that the SCENIC-ROUTE problem is in **NP**. **[20 points]**
- b) Prove that the SCENIC-ROUTE problem is **NP**-complete. **[30 points]**

Problem 2. (50 points)

As CEO of Naboo starship manufacturing company Theed Hangars, you have spent the last few weeks scrambling between star systems to strike deals across the Galactic Republic's Free Trade Zone for relief from impending tariffs on inter-system trade, which threaten to devastate the N-1 starfighter supply chain. Unfortunately, it is becoming increasingly evident that you may not be able to keep the entire N-1 supply chain intact in the face of these new tariff schedules, even with the quota allowance deals which you have been able to strike.

Having spoken to RNSF officials about the matter, they have agreed to vary their order of record for the year, and thereby accept as many N-1 starfighters as your company produces for the year, if it will enable you to put in place a supply chain schedule which keeps as many factories open as possible. While this may help, the problem of arriving at a supply chain schedule which keeps as many factories open as possible may yet prove to be tricky.

Formally, an instance of the N1-SUPPLY-CHAIN problem consists of the following:

- A set of *star systems* numbered from 1 to m .
- For each pair of star systems i, j , and each $T \in \{\mathbf{frame}, \mathbf{engines}\}$, you are given a maximum allowable *inter-system quota*, a non-negative integer q_{ijT} indicating the maximum amount of goods of type T which are allowed to be imported from system i to system j .
- A set of n *factories*. For each factory i , you are given...
 - Its *type*, $T_i \in \{\mathbf{frame}, \mathbf{engines}, \mathbf{fitout}\}$.
 - Its *location*, a star system $L_i \in \{1, \dots, m\}$.
 - Its *required output to remain profitable*, a non-negative integer a_i
 - Its *maximum manufacturing capacity*, a non-negative integer b_i .

A *supply chain schedule* is an assignment of any amount of spaceframes to **frame**, **engines**, and **fitout** factories, such that

1. For any two distinct systems i and j , the amount of spaceframes passing from factories of type **frame** in system i to factories of type **engines** in system j is at most $q_{ij\mathbf{frame}}$.
2. For any two distinct systems i and j , the amount of spaceframes passing from factories of type **engines** in system i to factories of type **fitout** in system j is at most $q_{ij\mathbf{engines}}$.
3. The amount of spaceframes assigned to any factory i is at most b_i .
4. The amount of spaceframes assigned to any factory i is either *zero* (in which case factory i is considered *closed*), or at least a_i (in which case factory i is considered *open*).

The goal of the N1-SUPPLY-CHAIN problem is to determine a supply chain schedule such that that *as many factories as possible* remain open.

- a) Define the decision version of the N1-SUPPLY-CHAIN problem. [5 points]
- b) Prove that the decision version of the N1-SUPPLY-CHAIN problem is in NP. [15 points]
- c) Prove that the decision version of the N1-SUPPLY-CHAIN problem is NP-complete. [30 points]

Note that you may use, without further justification, the fact that **Assignment 3 problem 1** has a polynomial-time solution.

Advice on how to do the assignment

- Assignments should be typed and submitted as pdf (no pdf containing text as images, no handwriting).
- Start by typing your student ID at the top of the first page of your submission. Do **not** type your name.
- Submit only your answers to the questions. Do **not** copy the questions.
- You may use paragraphs, lists or itemized points, pseudocode, or any combination of the above to organize your algorithm description. **Do not include actual code in your submission.** (Penalties will apply!)
- When designing an algorithm or data structure, it might help you (and us) if you give a brief (1-2 sentence) description of the overall idea, and after that develop and elaborate on details. If your algorithm is **stated in pseudocode** or the details of the algorithm are complex, then a brief description of the overall idea is **required**. Remember that an assignment in this course is an exercise in **effectively presenting and communicating technical ideas** (i.e. not just "solving the problem"), and will be graded accordingly.
- Refrain from explaining "*why*" your algorithm does what it does in the algorithm description itself. The "*why*" belongs in your **proof**. Your algorithm description should be a lean, complete, and unambiguous statement of just "*what*" your algorithm is.
- If you use pseudocode in your algorithm description, *avoid* adhering to any particular pseudocode "standard". Each line should endeavour to be "prosaic" in describing the intended step, with mathematical notation interspersed to denote any objects or concepts which you feel cannot be conveyed clearly in prose. Treat the pseudocode in Kleinberg and Tardos as exemplary.
- In general, try to be **prosaic rather than syntactic** when describing your algorithm. Treat this as an advantage! You do not need to describe your algorithm to a compiler or interpreter, so you can focus on phrasing your description for ease of readability by a human reader.
- Be careful with giving multiple or alternative answers. If you give multiple answers, then we will give you marks only for "your worst answer", as this indicates how well you understood the question.
- You can use the material presented in the lectures or textbooks without proving it. You do not need to write more than necessary (see comment above).
- Unless otherwise stated, we always ask about worst-case analysis, worst-case running times, etc.
- If you use further resources (books, scientific papers, the internet,...) to formulate your answers, then add references to your sources and explain it in your own words. Only citing a source doesn't show your understanding and will thus get you very few (if any) marks. Copying from any source without reference is considered plagiarism.